ProctorShield — Production Scaling & Infrastructure Guide

Document Version: 1.0 **Date:** June 10, 2026 **Product:** ProctorShield (Proctoring Service Platform) **Target Scale:** 1–2 Lakh (100,000–200,000) Concurrent Users **Prepared for:** Production deployment planning

---

Table of Contents

1. [Executive Summary](#1-executive-summary)
1. [Current System Analysis](#2-current-system-analysis)
1. [Traffic Profile at Scale](#3-traffic-profile-at-scale)
1. [Target Architecture & Microservices](#4-target-architecture--microservices)
1. [System Design Concepts](#5-system-design-concepts)
1. [Production Server Configuration](#6-production-server-configuration)
1. [Implementation Guide](#7-implementation-guide)
1. [Monthly Cost Plan](#8-monthly-cost-plan)
1. [Phased Rollout Plan](#9-phased-rollout-plan)
1. [What Breaks First Today](#10-what-breaks-first-today)
1. [Appendix — Key Files & Checklists](#11-appendix--key-files--checklists)

---

1. Executive Summary

ProctorShield is a multi-tenant online exam proctoring platform built as a **modular monolith** with:

- **1 Node.js API** (`platform-api`) — Express, PostgreSQL, WebSockets
- **1 Python AI sidecar** — FastAPI, face/gaze/phone/identity detection
- **3 frontends** — Super Admin, Tenant Admin, Candidate Exam App
- **1 shared SDK** (`@proctorshield/sdk`) — Candidate experience, lockdown, recording, AI

The system is well-architected for **development and demo scale** (~50–200 concurrent sessions per API instance) but requires substantial infrastructure investment to support **1–2 lakh simultaneous exam takers**.

Key Recommendations

| Priority | Action | |----------|--------| | 1 | Direct-to-S3 uploads (stop routing media through API) | | 2 | Split WebSocket gateway + Redis pub/sub for horizontal scale | | 3 | Queue + batch writes for face checks and events | | 4 | Client-side AI as default; server AI async and sampled | | 5 | Enforce tenant quotas at session init | | 6 | Load test 150k sessions before exam day |

Cost Summary (USD / Month)

| Scenario | Monthly Cost | |----------|-------------| | Dev / Staging | $600 – $1,200 | | Production Standby (no mega exam) | $4,500 – $7,500 | | Mega Exam Month (150k concurrent) | $28,000 – $55,000 | | Mega Exam Month (optimized) | $15,000 – $28,000 |

---

2. Current System Analysis

2.1 Repository Structure

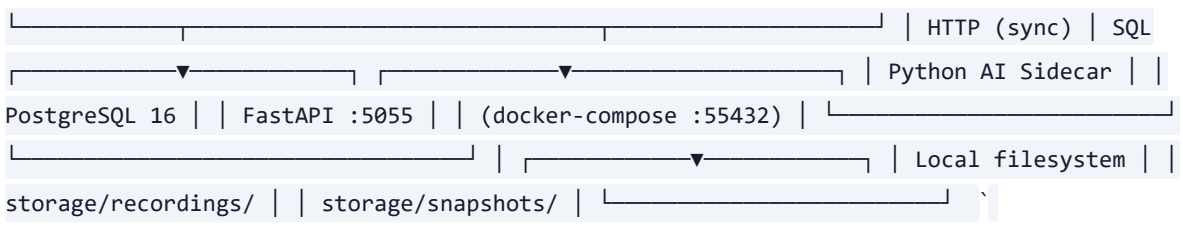
`` Procotring-servicea-app/ ├── apps/ | ├── super-admin-portal/ (Next.js 14, port 3100) | ├── tenant-admin-portal/ (Next.js 14, port 3200) | └── candidate-exam-app/ (Vite 5, port 3300) ├── services/ | └── platform-api/ (Express 4, port 3000) ├── packages/ | ├── sdk/ (@proctorshield/sdk) | ├── shared-types/ | ├── ui-components/ | └── utils/ ├── python/ (AI sidecar, port 5055) └── docker-compose.yml (Postgres only) ``

2.2 Tech Stack

| Layer | Technology | |-------|------------| | Monorepo | npm workspaces, Turbo 2 | | Frontend | Next.js 14, Vite 5, React 18, TypeScript | | Backend API | Express 4 (ESM), Node `http` + `ws` | | AI sidecar | Python 3.10+, FastAPI, MediaPipe, YOLO, ArcFace | | Database | PostgreSQL 16 (`pg` driver, pool `max: 20`) | | Auth | JWT, bcrypt, SHA-256 API key hashing | | Real-time | Native WebSocket (`ws`), WebRTC (browser P2P) | | Media | Browser MediaRecorder, chunked uploads, **local filesystem** | | Queues | **None** | | Caching | **None** (small in-process Maps only) | | Object storage | Local disk (`storage/`), `object_key` schema ready for S3 | | TURN/STUN | Google STUN default; dev placeholder for TURN |

2.3 Current Architecture Diagram

` ┌──┐ | Client Layer | | Super Admin (3100) | Tenant Admin (3200) | Candidate SDK (3300) | | └──────────────── @proctorshield/sdk ────────────────┘ | └──────────────────────────┬──────────────────────────────────┘ | HTTP / WebSocket / WebRTC ┌──────────────────────────▼──────────────────────────┐ | platform-api (Express monolith) | | /v1/superadmin /v1/tenant/* /v1/sdk/* /v1/media/* | | WebSocket: /v1/sdk/ws/sessions/{sessionId} | | In-memory: WS peers, AI proctor state, reference cache |



2.4 Current Capacity vs Target

Component	Current State	Max Capacity Today
platform-api	Single Express process, `pg` pool max 20	~50–200 concurrent sessions/instance
WebSockets	In-memory peer map	Cannot scale horizontally
AI	Sync HTTP to Python sidecar	~10–50 sessions per GPU
Media	Local disk	Single machine only
Database	Single Postgres, no replicas	Write-bound at ~1k writes/sec
Cache/Queue	None	No async processing
Quotas	`max_concurrent_sessions` in DB	**Not enforced at runtime**

2.5 What Already Works Well for Scale

- **Client-side face checks** in Web Worker (SDK README: safe for 50k+ candidates)
- **Chunked recording** (10–15s chunks)
- **AI throttle + circuit breaker** (`AiSidecarThrottle.ts`)
- **Deferred AI persistence** (non-blocking writes)
- **Partition-friendly schema** comments on `session_face_checks`
- **`object_key` columns** ready for S3/GCS
- **AI sidecar** as separate process (can scale independently once async)

---

3. Traffic Profile at Scale

3.1 Assumptions — 150,000 Concurrent Sessions

- Exam duration: 2–3 hours

- Live proctors: ~5% (7,500 proctors)
- Recording: Camera + screen, chunked upload
- Face checks: Every 10–15 seconds (client-side)

3.2 Peak Load Estimates

| Stream | Rate at Peak | |--------|--------------| | WebSocket connections | ~157,000 persistent (150k candidates + 7.5k proctors) | | Session init burst | 150k in 15–30 min ≈ **80–170 req/s** | | Face-check DB writes | **~15,000 writes/sec** (if every 10s, direct to DB) | | Server AI ticks (if 1 Hz) | **150,000 req/s — DO NOT do synchronously** | | Recording chunk uploads | **~25,000 uploads/sec** | | Storage ingress | **~12+ GB/s peak** (~86 TB in 2 hours) |

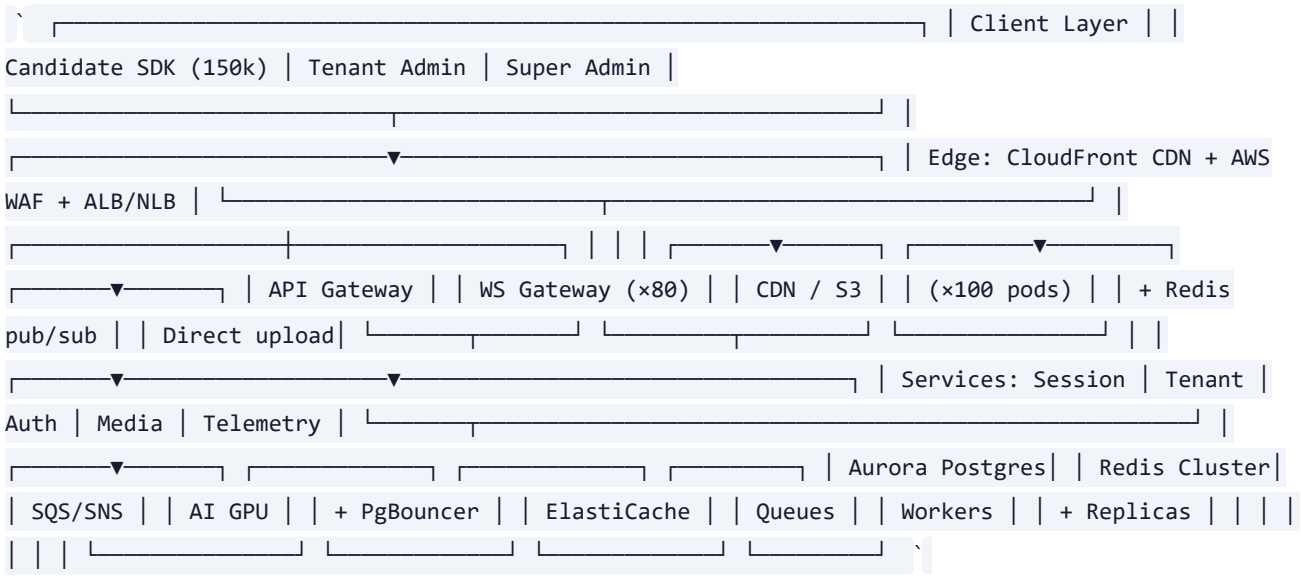
3.3 Critical Insight

At 150k scale, **the browser must do most AI work**. Server-side AI at 1 Hz for 150k users would need thousands of GPUs. Use client-side face checks as default; server AI should be **sampled** (1 tick per 30–60s) or **violation-triggered only**.

---

4. Target Architecture & Microservices

4.1 Target Production Architecture



4.2 Microservices to Build (Priority Order)

| # | Service | Responsibility | Split From | |---|---------|----------------|------------| | 1 | **Session Service** | init, config, end, status, pause/resume | platform-api SDK routes | | 2 | **WebSocket Gateway** | Chat, timer sync, WebRTC signaling | sdkSessionWs.js, chat.js | | 3 | **Media**

Service | Presigned uploads, merge jobs, signed URLs | recording.js, localStorage.js | | 4 | **Telemetry Service** | Face checks, violations, batched events | sdkEvents.js, monitoringPersistence.js | | 5 | **AI Orchestrator + Workers** | Queue-based inference | unifiedProctorAnalyze.js, Python sidecar | | 6 | **Auth Service** | JWT, API key lookup (cached), WS tokens | auth.js, sdkApiKey.js | | 7 | **Tenant Service** | CRUD, quotas, branding, users | Tenant admin routes | | 8 | **Webhook Worker** | Async delivery with retry/DLQ | webhooks.js | | 9 | **Analytics Service** | Aggregations, dashboards | analyticsService.js | | 10 | **Report Service** | PDF generation, async | media.routes.js | | 11 | **TURN Service** | Managed coturn or Twilio/Xirsys | turn-credentials endpoint | **Keep in monolith initially:** Super admin, policy templates, proctor assignments (low QPS).

4.3 Production Technology Stack

| Layer | Technology | |-------|------------| | **Compute** | Kubernetes (EKS/GKE), 100+ API pods, 80+ WS pods | | **Database** | Aurora PostgreSQL + PgBouncer + partitioning + read replicas | | **Cache** | Redis Cluster (ElastiCache) | | **Queue** | Kafka or AWS SQS + SNS | | **Storage** | S3 + CloudFront signed URLs | | **Real-time** | Dedicated WS gateway + Redis pub/sub | | **AI** | Client-first; async GPU workers behind queue | | **CDN/WAF** | CloudFront + AWS WAF | | **TURN** | coturn cluster or managed (Twilio/Xirsys) | | **Observability** | Prometheus, Grafana, OpenTelemetry, Sentry | | **IaC** | Terraform + Helm |

---

5. System Design Concepts

5.1 Stateless API + Stateful Real-Time Split

WebSocket peers are stored in memory today (chat.js). **Fix:** Dedicated WS gateway pods + Redis pub/sub for broadcastToSession . Use sticky sessions or session-ID hash routing.

5.2 Write-Behind / Event Sourcing for Hot Tables

session_face_checks at 15k writes/sec will crush Postgres. **Techniques:**

- SDK batches face checks (3–5 results per POST every 30s)
- API writes to Kafka/SQS → consumer bulk-inserts in batches of 500–1000
- Partition session_face_checks by created_at (monthly)
- Move hot telemetry to TimescaleDB or ClickHouse

5.3 Direct-to-Object-Storage Uploads

Never stream 25k uploads/sec through the API. **Flow:**

1. SDK requests presigned URL from Media Service
1. SDK uploads chunk directly to S3 (multipart)

1. SDK notifies API with object_key only (~200 bytes metadata)

1. Merge workers run async on session end

5.4 Caching Layer (Redis Cluster)

| Cache Key | TTL | Purpose | |-----------|-----|---------| | apikey:{hash} → tenant | 5–15 min | Avoid DB hit on every SDK request | | session:{id}:status | 30s | Status polls | | tenant:{id}:quota | 1 min | Enforce max_concurrent_sessions | | ws:session:{id}:peers | ephemeral | Cross-pod WS presence |

5.5 Connection Pooling (PgBouncer)

Current pool: max: 20 per API instance. At scale: PgBouncer in transaction mode, 100 API pods × 10 connections = 1000 → pooled to ~200–300 real Postgres connections.

5.6 Backpressure & Circuit Breakers

- API rate limits per tenant (token bucket in Redis)
- Admission control at session init
- Queue depth limits for AI workers
- Graceful degradation: skip server AI, reduce face-check frequency under load

5.7 Multi-Tenancy at Scale

- Shared DB with tenant_id filter (current model)

- For mega-tenants: dedicated shard or isolated Postgres cluster

- Enforce quotas at POST /v1/sdk/sessions/init

- Per-tenant rate limits and connection budgets

5.8 CDN + Static Assets

- SDK bundle on CloudFront (versioned, immutable cache)
- Admin portals on Vercel/Amplify or containerized
- Media playback via signed CloudFront URLs

5.9 Observability

- OpenTelemetry traces across API → queue → AI → DB
- Prometheus + Grafana: WS connections, queue lag, DB replication lag
- Sentry for client SDK errors (sample at 1–10% for 150k browsers)
- Structured audit logs → OpenSearch

5.10 Disaster Recovery

- Blue/green deploy freeze 48h before exam
- Chaos test: kill 30% of WS pods mid-exam
- Aurora Multi-AZ, tested failover
- Regional DR: secondary region with read replica promotion
- Synthetic monitoring: 10k fake sessions in staging 1 week before

---

6. Production Server Configuration

6.1 Recommended Cloud: AWS ap-south-1 (Mumbai)

Edge & Frontends

Resource	Spec	Count	Notes
CloudFront	Global	1 distribution	SDK JS, static assets
WAF	Rate rules	1	DDoS, bot protection
ALB	L7	2 (multi-AZ)	API + WS
Next.js admin apps	2 vCPU, 4GB	3–5 pods each	Auto-scale on CPU

API Plane

Resource	Spec	Count (peak)
EKS node group	c6i.2xlarge (8 vCPU, 16GB)	20–40 nodes
API pods	2 vCPU, 4GB each	**80–150 pods**
PgBouncer	Sidecar or dedicated	3

Sizing: 150k sessions × ~10 req/min ≈ 25k req/s → need **50–**

80 API pods minimum, plan **100+ with headroom.**

WebSocket Gateway

| Resource | Spec | Count | |----------|------|-------| | WS pods | 4 vCPU, 8GB | **60–100 pods** (~2k connections/pod) | | Redis Cluster | r6g.2xlarge × 6 shards | 1 cluster | | NLB | TCP pass-through | 1 | **Sizing:** 157k connections ÷ 2k per pod ≈ **80 WS pods** minimum.

Database

| Resource | Spec | |----------|------| | Postgres Primary | db.r6g.8xlarge or Aurora PostgreSQL io-optimized | | Read Replicas | 3–5 × db.r6g.4xlarge | | PgBouncer | 3 instances, max 300 server connections | | Partitioning | session_face_checks, session_events (monthly) |

Media (S3)

| Resource | Spec | |----------|------| | S3 bucket | Standard + Intelligent-Tiering (~100 TB per mega-exam) | | S3 Transfer Acceleration | Enabled | | Merge workers | 100–200 Lambda/ECS workers | | CloudFront | Origin = S3, signed URLs |

AI (If Server-Side Required)

| Resource | Spec | Count | |----------|------|-------| | GPU workers | g5.xlarge (1× A10G) | 200–500 at 1 Hz (avoid) | | GPU workers (optimized) | g5.xlarge | **50–100** at 0.2 Hz or violation-only | | AI queue | Kafka or SQS | — | **Recommendation:** Client-side face checks only during peak; server AI 1 sample per 60s or violation-triggered.

TURN (Live Proctoring)

| Resource | Notes | |----------|-------| | coturn on c6i.4xlarge × 10–20 | Or Twilio/Xirsys managed | | 7,500 proctors × 1 WebRTC stream | ~7.5 Gbps media relay at peak |

Async Workers

| Worker | Pods | Trigger | |--------|------|---------| | Webhook dispatcher | 20 | Kafka/SQS | | Recording merge | 50–100 | Session end event | | Report PDF | 10 | On demand | | Analytics ETL | 5 | Stream to ClickHouse |

6.2 Kubernetes Topology

` Region: ap-south-1 (Mumbai) — primary DR Region: ap-south-2 (Hyderabad) — warm standby Namespaces: proctorshield-edge (ingress, cert-manager) proctorshield-api (session, tenant, auth) proctorshield-realtime (ws-gateway) proctorshield-media (upload coordinator, merge workers) proctorshield-ai (orchestrator, gpu workers) proctorshield-workers (webhooks, reports) proctorshield-data (redis, pgbouncer) HPA rules: API: CPU 60% OR p99 latency > 200ms → scale WS:

`active_connections > 1500 per pod → scale AI: queue_depth > 1000 → scale GPU workers`

---

7. Implementation Guide

7.1 Repository Structure to Add

```
Procotring-servicea-app/
├── infrastructure/
│   ├── terraform/
│   │   ├── environments/dev/
│   │   ├── environments/prod/
│   │   └── modules/ (vpc, eks, aurora, elasticache, s3-cloudfront, sqs-sns, waf)
│   └── helm/
│       ├── proctorshield/
│       └── coturn/
├── services/
│   ├── platform-api/Dockerfile
│   ├── ws-gateway/ (NEW)
│   └── ai-worker/ (NEW)
└── .github/workflows/
    ├── build-api.yml
    └── deploy-prod.yml
```

7.2 Phase 0 — Containerize (Week 1)

Dockerfile for platform-api:

```dockerfile
FROM node:20-alpine AS deps
WORKDIR /app
COPY package.json package-lock.json ./
COPY services/platform-api/package.json services/platform-api/
RUN npm ci --workspace=@proctorshield/platform-api --omit=dev

FROM node:20-alpine
WORKDIR /app
ENV NODE_ENV=production
COPY --from=deps /app/node_modules ./node_modules
COPY services/platform-api ./services/platform-api
WORKDIR /app/services/platform-api
EXPOSE 3000
HEALTHCHECK CMD wget -qO- http://localhost:3000/health || exit 1
CMD ["node", "src/server.js"]
```

Split WebSocket: Extract sdkSessionWs.js into services/ws-gateway/ with Redis pub/sub replacing in-memory sessionPeers Map.

7.3 Phase 1 — Terraform Foundation (Week 2–3)

Modules to provision:

- VPC (3 public + 3 private subnets, NAT)
- EKS (API, WS, GPU node groups)
- Aurora PostgreSQL 16 (writer + 3 readers)
- ElastiCache Redis 7 (3 shards)
- S3 + CloudFront
- SQS + SNS (face-checks, events, webhooks, recording-merge)
- WAF on ALB

Deploy:

```bash
cd infrastructure/terraform/environments/prod
terraform init
terraform plan -out=plan.tfplan
terraform apply plan.tfplan
aws eks update-kubeconfig --name proctorshield-prod --region ap-south-1
```

7.4 Phase 2 — Database (Week 3)

- Deploy PgBouncer between API and Aurora
- Update DATABASE_URL to point at PgBouncer
- Run partition migration on session_face_checks, session_events
- Run migrations as K8s Job before each deploy

7.5 Phase 3 — Redis Cache (Week 4)

- Cache API key → tenant lookup (5–15 min TTL)
- Cache session status (30s TTL)
- Enforce max_concurrent_sessions at session init
- WS pub/sub for cross-pod broadcast

7.6 Phase 4 — S3 Media (Week 4–5)

New upload flow: `SDK → POST /recording-chunk/init → presigned S3 URL SDK → PUT directly to S3 SDK → POST /recording-chunk/complete { object_key, bytes, sequence } API → metadata row only`

Env vars: `env STORAGE_BACKEND=s3 S3_BUCKET=proctorshield-media-prod AWS_REGION=ap-south-1 CLOUDFRONT_DOMAIN=media.proctorshield.com`

7.7 Phase 5 — Queues (Week 5–6)

- API publishes face checks to SNS → SQS
- Telemetry worker bulk-inserts 500 rows at a time
- Webhook worker with retry + DLQ
- Recording merge worker on session end event

7.8 Phase 6 — Kubernetes + Helm (Week 6–7)

`bash helm upgrade --install proctorshield ./infrastructure/helm/proctorshield \ -f values-

prod.yaml \ --namespace proctorshield --create-namespace ` **HPA targets:**

- API: 20 min → 120 max replicas
- WS: 30 min → 100 max replicas
- AI workers: 5 min → 50 max (spot instances)

7.9 Phase 7 — Observability (Week 7–8)

- OpenTelemetry in API and WS gateway

- Prometheus /metrics endpoint

- kube-prometheus-stack Helm chart
- Sentry for SDK (sampled) and API
- Dashboards: active sessions, WS connections, queue depth, Aurora lag

7.10 Phase 8 — TURN (Week 8)

Option A (fastest): Twilio Network Traversal or Xirsys managed **Option B:** Self-hosted coturn on EKS behind NLB (UDP required)

7.11 Environment Variables (Production Secret)

```
env DATABASE_URL=postgresql://app:***@pgbouncer:6432/proctorshield
DATABASE_READ_URL=postgresql://app:***@pgbouncer-read:6432/proctorshield JWT_SECRET=<64-char
random> REDIS_URL=rediss://elasticache-endpoint:6379 STORAGE_BACKEND=s3 S3_BUCKET=proctorshield-
media-prod AWS_REGION=ap-south-1 SNS_FACE_CHECKS_ARN=arn:aws:sns:...
SQS_FACE_CHECKS_URL=https://sqs... AI_SERVICE_URL=http://ai-worker:5055 AI_MODE=async
PUBLIC_WS_BASE_URL=wss://ws.proctorshield.com PUBLIC_API_BASE_URL=https://api.proctorshield.com
TURN_PROVIDER=xirsys OTEL_EXPORTER_OTLP_ENDPOINT=http://otel-collector:4318 SENTRY_DSN=https://...
```

7.12 Load Testing (Before Exam Day)

Use k6 or Locust:

- Ramp 150k sessions over 15 minutes
- Hold 2 hours
- Measure: session init p99 < 500ms, WS connect > 99.9%, zero OOM kills, SQS age < 30s, Aurora CPU < 70%

---

8. Monthly Cost Plan

8.1 Three Plans

| Plan | Use Case | Monthly Total (USD) | |---|---|---| | **Plan A — Dev/Staging** | Build, test, pilots | $600 – $1,200 | | **Plan B — Production Standby** | Live platform, low traffic | $4,500 – $7,500 | | **Plan C — Mega Exam Month** | 1 exam, 150k concurrent | $28,000 – $55,000 | | **Plan C Optimized** | Spot, client AI, lifecycle policies | $15,000 – $28,000 |

8.2 Plan C — Complete Service Breakdown

| # | Service | Spec | Monthly Cost (USD) | |---|---|---|---| | 1 | EKS control plane | 1 cluster | $73 | | 2 | EC2 — API nodes | 40× c6i.2xlarge × 5 days + 8× rest | $4,500 – $7,000 | | 3 | EC2 — WS gateway | 60× c6i.2xlarge × 5 days + 6× rest | $4,000 – $6,500 | | 4 | EC2 — Workers | 30× c6i.xlarge spot × 5 days | $800 – $1,500 | | 5 | EC2 — GPU (AI) | 20× g5.xlarge spot × 3 days | $1,200 – $2,500 | | 6 | Aurora PostgreSQL | Writer + 3 readers | $3,500 – $6,000 | | 7 | ElastiCache Redis | 3 shards × 2 replicas | $1,800 – $2,800 | | 8 | S3 — media storage | 60–100 TB | $1,400 – $2,500 | | 9 | S3 — requests | ~200M PUT + 50M GET | $800 – $1,500 | | 10 | CloudFront | 5–15 TB egress | $400 – $1,200 | | 11 | Data transfer | 10–30 TB egress | $1,000 – $3,500 | | 12 | ALB + NLB | 2 ALB + 1 NLB | $150 – $400 | | 13 | NAT Gateway | 3 AZ | $350 – $900 | | 14 | AWS WAF | 1 Web ACL | $100 – $300 | | 15 | SQS + SNS | ~500M messages | $200 – $500 | | 16 | TURN / WebRTC | Managed or coturn | $2,000 – $8,000 | | 17 | Route 53 | 3–5 zones | $10 – $30 | | 18 | CloudWatch Logs | 500 GB – 2 TB | $250 – $800 | | 19 | Sentry | Business, sampled | $80 – $300 | | 20 | Backup (Aurora) | Daily snapshots | $200 – $500 | | 21 | Frontends | Vercel or EKS | $20 – $200 | | 22 | Misc (ECR, Secrets, SMS) | — | $100 – $300 | | | **TOTAL** | | **$28,000 - $55,000** |

8.3 Plan B — Production Standby

| Category | Monthly Cost | |---|---| | EKS + 6–8 EC2 nodes | $1,500 – $2,500 | | Aurora (1 writer + 1 reader) | $800 – $1,500 | | Redis (1 shard) | $400 – $700 | | S3 (5–10 TB) | $120 – $250 | | CloudFront + ALB + NAT + WAF | $300 – $600 | | Observability + misc | $200 – $500 | | **TOTAL** | **$4,500 - $7,500** |

8.4 Cost Drivers (Mega Exam Month)

| Rank | Service | % of Total | |---|---|---| | 1 | EC2 / EKS compute | ~35% | | 2 | S3 video storage | ~15% | | 3 | TURN / WebRTC relay | ~12% | | 4 | Aurora PostgreSQL | ~12% | | 5 | Data transfer | ~10% | | 6 | Redis | ~6% | | 7 | GPU (if used) | ~5% |

8.5 Storage Detail (150k Exam, 2 Hours)

| Item | Volume | Cost (90-day retention) | |---|---|---| |

Recordings (camera + screen) | 60–120 TB | $1,400 – $2,800 | | Snapshots | 2–5 TB | $50 – $120 | | Playback egress (10% reviewed) | 6–12 TB | $650 – $1,300 | **Lifecycle savings:** Standard → IA (day 31) → Glacier (day 90) cuts storage ~40–80%.

8.6 TURN Cost Options

Option	Exam-Day Cost	Best For
Twilio Network Traversal	$2,000 – $5,000	Fastest to production
Xirsys managed	$1,500 – $4,000	Medium scale
Self-hosted coturn	$500 – $1,500 instances + $2k–5k egress	Full control
No live video (chat only)	$0	Maximum savings

8.7 Cost Optimization (40–50% Savings)

Technique	Savings
Spot instances for workers, merge, AI	–30% compute
Client-only AI (no server GPU)	–$1,500–2,500
S3 lifecycle → Glacier	–40% storage
Camera only (no screen recording)	–50% storage
Chunk interval 12s → 30s	–20% upload costs
No live proctoring video	–$2,000–8,000 TURN
Reserved instances for base infra	–25% standby

8.8 Annual Outlook

Pattern	Annual Cost (USD)
12 months standby only	$54,000 – $90,000
10 standby + 2 mega exam months	$115,000 – $175,000
10 standby + 2 optimized exam months	$90,000 – $130,000

8.9 Recommended Monthly Budget

Scenario	Budget to Allocate
1 mega exam per month	**$30,000 – $45,000**
Standby only (no exam)	**$5,000 – $8,000**

---

9. Phased Rollout Plan

Phase 0 — Before Any Mega Exam (2–4 weeks)

1. `Enforce` max_concurrent_sessions `at session init`

1. Redis for API key cache + session status

1. S3 presigned uploads (remove local disk)

1. PgBouncer + per-pod pool config
1. Batch face-check inserts

Phase 1 — Horizontal Scale (1–2 months)

1. Extract WebSocket Gateway + Redis pub/sub
1. Split Media Service (presigned URLs, async merge)
1. ALB + 10–20 API replicas
1. Postgres read replica for admin/analytics
1. CDN for SDK bundle

Phase 2 — Event-Driven (2–3 months)

1. Kafka/SQS for face checks, events, webhooks
1. Telemetry Service with bulk writers
1. Partition hot tables
1. AI queue + GPU worker pool (async only)
1. Full observability stack

Phase 3 — Mega Exam Ready (3–6 months)

1. Load test 150k synthetic sessions
1. Dedicated infra for single mega-tenant
1. Multi-region DR drill
1. TURN cluster for live proctoring
1. Runbook + 24×7 war room for exam day

Week-by-Week First Steps

`| Day | Action | |-----|--------| | 1 | Add Dockerfile for platform-api; verify /health | | 2 | Terraform VPC + EKS (dev, small) | | 3 | Aurora dev cluster; run migrations; deploy 1 API pod | | 4 | S3 presigned URL endpoint for recording chunks | | 5 | ElastiCache Redis; cache API key lookup | | 6 | Extract ws-gateway; Redis pub/sub; deploy 2 WS pods | | 7 | Helm chart + HPA; CI pipeline to ECR |`

---

10. What Breaks First Today

`If deployed at 150k without changes:`

1. **Postgres** — 20-connection pool, 15k writes/sec on face checks

1. **In-memory WebSocket map** — second API instance breaks chat/signaling
1. **Local disk storage** — fills in minutes, not shared across pods
1. **Sync AI HTTP** — API threads blocked waiting on Python
1. **No admission control** — cascade failure when all 150k accepted
1. **API key DB lookup** — every SDK request hits Postgres
1. **WebRTC TURN** — Google STUN only; no TURN at scale for live proctors

---

11. Appendix — Key Files & Checklists

11.1 Key Codebase Files

| Concern | Path | |---|---| | API entry | services/platform-api/src/server.js | | Route registration | services/platform-api/src/app.js | | WebSocket | services/platform-api/src/ws/sdkSessionWs.js | | In-memory chat/peers | services/platform-api/src/services/tenant/chat.js | | DB config (pool max 20) | services/platform-api/src/config/database.js | | Local storage | services/platform-api/src/storage/localStorage.js | | AI analysis | services/platform-api/src/services/ai/unifiedProctorAnalyze.js | | AI sidecar | python/run_ai_server.py | | SDK | packages/sdk/src/ | | Migrations | services/platform-api/src/db/migrations/ | | Docker (Postgres only) | docker-compose.yml |

11.2 SDK Tuning for 150k Users

| Setting | Recommendation | |---|---| | faceCheckIntervalMs` | 15–30s during peak | | Server AI | Disable or 60s+ interval | | Recording chunk size | 20–30s | | Face check batching | 3–5 results per API call | | Web Worker | Keep — never move to main thread |

11.3 Production Readiness Checklist

- [ ] Dockerfile for API, WS gateway, AI worker
- [ ] Terraform modules (VPC, EKS, Aurora, Redis, S3, SQS)
- [ ] Helm charts with HPA
- [ ] S3 storage adapter (replace localStorage)
- [ ] Redis cache for API keys and WS pub/sub
- [ ] Queue-based face check writes
- [ ] PgBouncer deployment
- [ ] Table partitioning migration
- [ ] Quota enforcement at session init
- [ ] Presigned URL upload flow in SDK

- [ ] TURN provider (managed or coturn)
- [ ] CloudFront for SDK and media
- [ ] WAF rate limiting
- [ ] OpenTelemetry + Prometheus + Sentry
- [ ] CI/CD to ECR + Helm deploy
- [ ] Load test 150k sessions
- [ ] DR failover drill
- [ ] Exam-day runbook

11.4 Minimum Production Stack Summary

| Layer | Technology | |-------|------------| | Compute | EKS, 100+ API pods, 80+ WS pods | | Database | Aurora PostgreSQL + PgBouncer + partitioning + replicas | | Cache | Redis Cluster (ElastiCache) | | Queue | AWS SQS + SNS (or Kafka) | | Storage | S3 + CloudFront signed URLs | | Real-time | WS gateway + Redis pub/sub | | AI | Client-first; async GPU workers | | CDN/WAF | CloudFront + AWS WAF | | TURN | coturn or Twilio/Xirsys | | Observability | Prometheus, Grafana, OpenTelemetry, Sentry | | IaC | Terraform + Helm |

Document Information

This guide consolidates the complete ProctorShield production scaling analysis, implementation roadmap, and cost planning for deployment at **1–2 lakh (100,000–200,000) concurrent users**. **Disclaimer:** All cost figures are estimates in USD for AWS ap-south-1 (Mumbai) based on typical pricing. Actual costs depend on exam duration, recording policy, live proctoring ratio, data retention, and traffic patterns. Use AWS Pricing Calculator for firm quotes before procurement.

ProctorShield Production Scaling Guide v1.0 — June 2026
